

# Physical Intelligence $\pi_{0.5}$ — Unified Engine v1

Apex Compute

## 1 The $\pi_{0.5}$ Model

$\pi_{0.5}$  is a vision–language–action (VLA) policy of roughly three billion parameters that maps camera observations and a language instruction directly to robot actions. Each inference emits a  $10 \times 7$  action chunk: ten future control steps, each a 7-dimensional command giving the robot’s degrees of freedom (arm motion plus gripper). The computation splits into three stages run in sequence—a vision encoder, a language prefix, and a flow-matching action expert.

### 1.1 Vision encoder

Each image is encoded independently by a SigLIP ViT;  $\pi_{0.5}$  takes up to three slots (scene, wrist, one padded). Every  $224 \times 224$  image becomes 256 patch tokens of width 1152, run through  $L_v=27$  bidirectional layers (16 heads, head-dim 72, MLP width 4304). Re-run per image and never cached, it costs  $\approx 656$  GFLOP.

### 1.2 Language prefix

The patch tokens are projected to  $d=2048$ , concatenated with the instruction tokens, and processed once by a Gemma-2B backbone:  $L_p=18$  layers of grouped-query attention (8 query, 1 KV head, head-dim 256), RMSNorm, RoPE, and a gated SiLU MLP ( $I=16384$ ). At prefill length  $M_p=576$  one layer costs

$$2M_p d d_q + 4M_p d d_{kv} + 2M_p d_q d + 4M_p^2 d_q + 6M_p d I, \quad (1)$$

( $d_q=2048$ ,  $d_{kv}=256$ ), dominated by the MLP; over 18 layers the prefix is  $\approx 2332$  GFLOP (76% of the model). Each layer’s K/V is cached and reused by every denoise step.

### 1.3 Action expert

A second, smaller Gemma ( $\sim 300$  M;  $H=1024$ , MLP 4096, 18 layers) turns noise into the action chunk by flow matching: from Gaussian noise it integrates a learned velocity field  $v_\theta$  over  $N=10$  Euler steps ( $\Delta t = -0.1$ ),

$$x_{t+\Delta t} = x_t + \Delta t v_\theta(x_t, t, \text{KV}_{\text{prefix}}), \quad (2)$$

each step attending over the cached prefix K/V and reading out actions through a  $(1024 \rightarrow 32)$  head. At only  $\approx 71$  GFLOP (2.3%) it is arithmetically small but the sole sequential stage, and its per-step error accumulates through the integrator.

Table 1: Per-stage parameter and FLOP budget (per inference). *Effective* uses true model dims; *FPGA* is the padded shape the 64-wide SIMD ALU actually issues. The prefix is already 64-aligned (no padding); the vision tower pads only its attention head (72→128, +2.9%); the action expert pads its query horizon 10→64 and action width 32→64, inflating its work  $\sim 6.5\times$  — so overall the FPGA issues  $\sim 13\%$  more FLOPs than the model nominally needs.

| Stage                          | Params                         | Eff. GFLOP    | FPGA GFLOP    |
|--------------------------------|--------------------------------|---------------|---------------|
| Vision tower (SigLIP, 3 slots) | $\sim 0.41$ B                  | 655.9         | 674.9         |
| Language prefix (Gemma-2B)     | $\sim 2.0$ B                   | 2332.0        | 2332.0        |
| Action expert (Gemma-300M)     | $\sim 0.31$ B                  | 71.0          | 459.4         |
| <b>Total</b>                   | <b><math>\sim 2.7</math> B</b> | <b>3058.9</b> | <b>3466.3</b> |

## 2 Porting

The Unified Engine v1 is a soft accelerator IP block that can be deployed on any sufficiently large FPGA; it operates on 64-element alignment. In this work it is targeted to a Xilinx Kintex-7 with 4 GB of attached DRAM. These two constraints—64-alignment and the 4 GB DRAM budget—drive the porting transformations below.

**IF4 weights / DRAM budget.** The  $\sim 13$  GB bf16 checkpoint does not fit, and writes at/above the 4 GB boundary fail on this board. All matmul weights use 4-bit block quantization (block 64, weight-only), shrinking the three stacks to  $\sim 1.56$  GB. Weights are dequantized to bf16 immediately before each multiply-accumulate, so IF4 is a *footprint enabler*, not a *speedup* — compute is bf16 throughout.

**Vision head-dim pad.** Head-dim 72 is not 64-/128-aligned, so the attention matmuls are padded 72→128 and contracted back via a precomputed  $128\times 72$  selection matrix; the  $Q/K/V/O$  projections stay at 72.

**Prefix masking.** The masked image slot is dropped, compressing prefill 832→576 (bit-identical,  $\sim 24\%$  faster). Rotary positions use cumulative-sum-of-mask indices, not `arange`, so dropping a slot does not shift phase. The non-causal/causal mixed attention mask is supplied as a static bias tensor in DRAM.

**Denoise loop.** All  $10\times 18$  steps are unrolled into one program issued with a single execute call; AdaRMSNorm conditioning is a matmul to  $3H$  split into scale/shift/gate. Every strided SRAM→DRAM copy needs an explicit per-index destination offset — a missing one yields finite-but-scrambled output (not a NaN), and fixing these recovered the expert from  $-10$  to  $\sim 29$  dB.

## 3 Quantization

### 3.1 The Quantization Floor

IF4 is the accelerator’s 4-bit weight format: a per-64-block hybrid of signed INT4 (uniform grid  $[-8, 7]$ ) and FP4 E2M1 (non-uniform codebook  $\{0, \pm 0.5, \pm 1, \pm 1.5, \pm 2, \pm 3, \pm 4, \pm 6\}$ , finer near zero). At pack time each block is quantized *both* ways and the lower-MSE variant kept; the choice rides in the *sign* of the block’s scale (neg → INT4, pos → FP4), so the on-chip dequantizer picks the decode path from one bit, inline before the MAC—no metadata. INT4 suits dense blocks, FP4 the peaky/outlier ones. (Results below use the INT4 variant, `q4_64`.)

We measure the accuracy cost against a bf16 reference (identical torch run, same inputs/noise, diffed per stage; Table 2,  $\text{SNR}=20\log_{10}(\|b\|/\|a-b\|)$ , masked rows excluded, sample 0)—the theoretical quantization floor, isolated from any hardware effect. Vision is nearly untouched (25.4 dB); the prefix caches degrade with depth, V worse than K (10.7 vs. 16.0 dB mean); and denoise carries the largest loss as per-step error accumulates through Eq. (2)— $x_t$  walks  $45 \rightarrow 16.3$  dB (Table 3), concentrating in low-magnitude action dims (dim 1, 1.5 dB) while task-critical dims hold (dim 0 24.9, gripper 40.7 dB). SNR is a diagnostic; the acceptance gate is the closed-loop LIBERO agreement (Sec. 3.2).

Table 2: IF4 vs. bf16 per section (torch reference, sample 0).

| Model section                     | SNR (dB)     | Range (dB)                |
|-----------------------------------|--------------|---------------------------|
| Vision encoder (3 cameras)        | 25.37        | 24.3–26.6                 |
| Prefix K-cache (18 layers)        | 15.96        | 12.5–19.6                 |
| Prefix V-cache (18 layers)        | 10.65        | 7.0–15.4                  |
| Denoise velocity $v_t$ (10 steps) | 23.49        | 11.5–26.2                 |
| <b>End-to-end action (10×7)</b>   | <b>16.31</b> | MSE $4.05 \times 10^{-3}$ |

Table 3: Final-action detail: per-dimension SNR (left) and error accumulation of the running state  $x_t$  across Euler steps (right). Sample 0.

| Action dim      | SNR (dB) | Euler step | $x_t$ SNR (dB) |
|-----------------|----------|------------|----------------|
| dim 0 (primary) | 24.91    | step 0     | 44.95          |
| dim 1           | 1.50     | step 3     | 30.13          |
| dim 2           | 10.08    | step 5     | 24.15          |
| dim 3           | 6.73     | step 7     | 20.53          |
| dim 4           | 4.65     | step 8     | 19.26          |
| dim 5           | 17.61    | step 9     | 16.31          |
| dim 6 (gripper) | 40.73    | (final)    | 16.31          |

### 3.2 LIBERO

LIBERO is a standard tabletop manipulation benchmark for evaluating language-conditioned robot policies: a suite of pick-and-place and articulated tasks in simulation, scored by closed-loop task success over many episodes. Its observation uses **two** camera views—a fixed scene camera and a wrist-mounted camera.  $\pi_{0.5}$ , however, exposes **three** image slots, and we process all three: the two real LIBERO cameras plus a third all-zero (attention-masked) slot. Encoding the empty slot is intentional—other embodiments and benchmarks drive the policy with a genuine third camera for their robot actions, so keeping the full 3-slot path exercised on hardware makes the port drop-in for those settings at no correctness cost (the masked slot contributes nothing to the output).

**An episode is many inferences.** Each model call in Sec. 1 predicts a  $10 \times 7$  chunk—ten future control steps. LIBERO runs *closed-loop*: the simulator executes the first  $k=10$  steps of that chunk, then feeds the resulting new observation back to the policy for a fresh chunk, and repeats. So a single task *episode* is a *sequence* of inferences—one every  $k$  sim steps—not a one-shot prediction, and the episode runs until the task’s goal predicate fires (success) or a per-suite step cap is hit (failure). An episode therefore takes  $\lceil \text{steps}/k \rceil$  inferences: the successful runs in Table 4 are  $\sim 10$ –16 inferences, the capped failures  $\sim 23$ –29. This is what makes the closed-loop test meaningful

and also much heavier than a single forward pass—every one of those inferences runs the full vision + prefix + 10-step denoise pipeline on hardware.

Per-tensor SNR is only a diagnostic; the real acceptance test is whether the quantized policy *acts* the same across that whole loop. Table 4 is the closed-loop comparison—the FPGA IF4 policy vs. the CUDA bf16 reference over 10 prompts (9 `libero_spatial` + 1 `libero_object`), one episode each, with identical seed and fixed denoise noise so both platforms see byte-identical observations. We score each by task success and by the per-episode step count.

Table 4: LIBERO closed-loop agreement: FPGA (IF4) vs. CUDA (bf16), 10 prompts, 1 episode each, fixed seed/noise. ✓=success, ×=failure; the number is episode length in sim steps.  $\Delta$  is FPGA – CUDA steps on successes (both failures run to the step cap, so  $\Delta$  is not meaningful there). The two policies reach the *same* outcome on every prompt.

| Prompt      | FPGA (IF4)  | CUDA (bf16) | $\Delta$ steps | Agree        |
|-------------|-------------|-------------|----------------|--------------|
| 0           | ✓ 94        | ✓ 95        | −1             | ✓            |
| 1           | ✓ 160       | ✓ 126       | +34            | ✓            |
| 2           | ✓ 123       | ✓ 115       | +8             | ✓            |
| 3           | ✓ 108       | ✓ 109       | −1             | ✓            |
| 4           | × 230       | × 230       | —              | ✓            |
| 5†          | × 290       | × 290       | —              | ✓            |
| 6           | ✓ 121       | ✓ 120       | +1             | ✓            |
| 7           | × 230       | × 230       | —              | ✓            |
| 8           | ✓ 107       | ✓ 106       | +1             | ✓            |
| 9           | × 230       | × 230       | —              | ✓            |
| <b>Rate</b> | <b>6/10</b> | <b>6/10</b> |                | <b>10/10</b> |

†Prompt 5 is a `libero_object` task; the original `libero_spatial` task 5 exercised a denoise-loop compile path that crashed the FPGA bring-up, so it is substituted with the next unused prompt.

**Takeaway.** IF4 is not just numerically close—it is behaviorally identical. Across all 10 prompts the FPGA IF4 policy and the bf16 reference reach the *same* success/failure outcome (10/10 agreement, 6/6 shared successes), and on the successful episodes their trajectory lengths differ by only a handful of steps ( $\Delta$  within a few percent, +34 on one longer reach). The 16.3 dB end-to-end SNR floor of Sec. 3 sounds alarming in isolation, but the low-SNR components are small-magnitude action dimensions that do not change what the robot *does*—the task-critical axes and gripper are preserved, and the closed-loop behavior confirms it.

## 4 Throughput: FPGA vs. CUDA

We time each stage on both platforms and report it in the same form: per-section wall-clock and the achieved rate (section FLOPs  $\div$  time). FPGA times come from the engine’s own instrumentation. The GPU baseline is openpi’s JAX/XLA Pi0 — the deployment path, sustaining 130.2 ms/inference (7.68 Hz), batch 1 — broken into the same three stages by jitting each separately and forcing device completion (`block_until_ready`) at the boundaries. Because these stages are dispatched separately rather than fused into the deployed `lax.while_loop`, their sum (135.4 ms) slightly exceeds the fused end-to-end (130.2 ms); the per-stage split is used for the breakdown, the fused number for the headline.

Table 5: RTX 5070 (bf16, openpi JAX/XLA) per-section timing and achieved throughput. Batch 1, 10 denoise steps, sample 0. Per-stage times are separately-jitted; End-to-end is the fused `policy.infer`.

| Model section             | Time (ms) | GFLOP/s |
|---------------------------|-----------|---------|
| Vision encoder            | 19.41     | 33 793  |
| Prefix (Gemma-2B)         | 79.29     | 29 411  |
| Action expert (denoise)   | 36.70     | 1 935   |
| <b>End-to-end</b>         | 130.15    | 23 502  |
| <b>Throughput (inf/s)</b> | 7.68      |         |

Table 6: FPGA (IF4) per-section timing and achieved throughput (Unified Engine v1 on Xilinx Kintex-7). Batch 1, 10 denoise steps, sample 0, 3-slot vision, 832-token prefix. GFLOP/s and % peak are on the hardware-issued (padded) FLOPs; theoretical peak is 26.6 GFLOP/s (64 MACs  $\times$  2  $\times$  208 MHz).

| Model section             | Time (s)     | Share | Eff. GFLOP    | HW GFLOP      | GFLOP/s (hw) | % peak     |
|---------------------------|--------------|-------|---------------|---------------|--------------|------------|
| Vision encoder            | 35.4         | 18.3% | 655.9         | 850.7         | 24.0         | 90%        |
| Prefix (Gemma-2B)         | 137.6        | 71.1% | 3399.8        | 3399.8        | 24.7         | 93%        |
| Action expert (denoise)   | 20.6         | 10.6% | 74.7          | 483.5         | 23.5         | 88%        |
| <b>End-to-end</b>         | <b>193.6</b> | 100%  | <b>4130.4</b> | <b>4734.0</b> | <b>24.5</b>  | <b>92%</b> |
| <b>Throughput (inf/s)</b> |              |       |               | 0.0052        |              |            |

Table 7: Silicon (IF4, *estimated*) per-section timing and achieved throughput — projected ASIC implementation of the Unified Engine v1 IP: 8 engines at 1.6 GHz (64 MACs each; 1638.4 GFLOP/s aggregate peak), 100 GB/s DRAM. Batch 1, 10 denoise steps. Linear 61.5 $\times$  compute scaling of the measured FPGA run at its achieved %-of-peak; no section is bandwidth-bound (see below).

| Model section             | Time (ms) | GFLOP/s |
|---------------------------|-----------|---------|
| Vision encoder            | 575       | 1 479   |
| Prefix (Gemma-2B)         | 2 236     | 1 520   |
| Action expert (denoise)   | 335       | 1 444   |
| <b>End-to-end</b>         | 3 146     | 1 505   |
| <b>Throughput (inf/s)</b> | 0.32      |         |

**Why the projection is linear.** Going from one engine at 208 MHz to 8 engines at 1.6 GHz multiplies compute by 61.5 $\times$  (26.6  $\rightarrow$  1638.4 GFLOP/s peak) but DRAM bandwidth by only 16.7 $\times$  ( $\sim$ 6  $\rightarrow$  100 GB/s), so each section survives as compute-bound only if its arithmetic intensity clears the ASIC ridge point of 1638.4/100  $\approx$  16 FLOP/byte. Traffic here is dominated by streaming the IF4 weights ( $\sim$ 0.53 B/param): the prefix reads  $\sim$ 1.1 GB once for its 3399.8 GFLOP ( $\sim$ 3200 FLOP/byte), the vision tower  $\sim$ 0.65 GB over three slots ( $\sim$ 1300 FLOP/byte), and the worst case — the action expert, whose  $\sim$ 0.16 GB are re-streamed on *every* denoise step against just  $\sim$ 48 GFLOP of padded work per step — still lands at  $\gtrsim$ 200 FLOP/byte with activation and KV-cache traffic included. All three sections sit an order of magnitude above the ridge (worst-section demand  $<$ 10 GB/s of the 100 available), so no stage becomes bandwidth-bound and times extrapolate linearly at the FPGA’s measured 88–93% of peak, assuming ideal 8-way engine scaling: 3.15 s per inference (0.32 inf/s), 61.5 $\times$  the FPGA, within 24 $\times$  of the RTX 5070’s 7.68 inf/s.

**Config.** FPGA: Unified Engine v1 IP on Xilinx Kintex-7 at 208 MHz (26.6 GFLOP/s theoretical peak), IF4 weights / bf16 compute, 4 GB DRAM. GPU: RTX 5070, bf16, 12 GB, openpi JAX/XLA. Both: batch 1, 10 denoise steps, sample 0.

The entire  $\pi_{0.5}$  VLA — vision, Gemma-2B prefix, and the 10-step flow-matching expert — runs on the Unified Engine v1. The recurring theme is the split between where FLOPs concentrate (the compute-bound prefix) and where latency does (the sequential, FLOP-light denoise loop), and how IF4 trades a controlled, section-localized accuracy cost for fitting a  $\sim 13$  GB model in a sub-4 GB budget. Tables 1–7 complete the characterization.